

```
chandrika_battina@DESKTOP-US3G0PF:~$ ls
ANU      file1.txt  fork1     fork3     hello     hello.c.save.2  pid.c     pipes     program1.c  s2sampleProgram.c  signo.c
Skill    fork      fork1.c   fork3.c   hello.c   input.txt       pid.c.save  pipes.c    program2.c  s2sampleProgram.c.save  week1
Week1.c  fork.c    fork2     fork4.c   hello.c.save  output.txt      pid.c.save.1  process.c  program2.c  shell               signo
example.txt  fork.c.save  fork2.c   forkk.c   hello.c.save.1  pid            pid.c.save.2  program1  s2
chandrika_battina@DESKTOP-US3G0PF:~$ nano address.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc address.c -o address
chandrika_battina@DESKTOP-US3G0PF:~$ ./address
Value of x = 10
Virtual address of x = 0x7ffd1cbc88e4
chandrika_battina@DESKTOP-US3G0PF:~$
```

```
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

```
chandrika_battina@DESKTOP-US3G0PF:~$ ls
ANU      file1.txt  fork1     fork3     hello     hello.c.save.2  pid.c     pipes     program1.c  s2sampleProgram.c  signo.c
Skill    fork      fork1.c   fork3.c   hello.c   input.txt       pid.c.save  pipes.c    program2.c  s2sampleProgram.c.save  week1
Week1.c  fork.c    fork2     fork4.c   hello.c.save  output.txt      pid.c.save.1  process.c  program2.c  shell               signo
example.txt  fork.c.save  fork2.c   forkk.c   hello.c.save.1  pid            pid.c.save.2  program1  s2
chandrika_battina@DESKTOP-US3G0PF:~$ nano address.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc address.c -o address
chandrika_battina@DESKTOP-US3G0PF:~$ ./address
Value of x = 10
Virtual address of x = 0x7ffd1cbc88e4
chandrika_battina@DESKTOP-US3G0PF:~$
```

```

#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address   : %p\n", (void *)main);
    printf("Data address    : %p\n", (void *)&global);
    printf("BSS address     : %p\n", (void *)&bss_variable);
    printf("Heap address     : %p\n", (void *)heap);
    printf("Stack address    : %p\n", (void *)&local);

    free(heap);

    return 0;
}

```

```

Virtual address of x = 0x7ffde000
chandrika_battina@DESKTOP-US3G0PF:~$ nano memory.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc memory.c -o memory
chandrika_battina@DESKTOP-US3G0PF:~$ ./memory
Code address   : 0x5fe3d355f1a9
Data address    : 0x5fe3d3562010
BSS address     : 0x5fe3d3562018
Heap address    : 0x5fe412dcd010
Stack address   : 0x7ffe2637c1fc
chandrika_battina@DESKTOP-US3G0PF:~$ nano page.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc page.c -o page
chandrika_battina@DESKTOP-US3G0PF:~$ ./page
Enter page number (0-2):

```

```

#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}

```

```

Stack address: 0x7ff2037c1fc
chandrika_battina@DESKTOP-US3G0PF:~$ nano page.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc page.c -o page
chandrika_battina@DESKTOP-US3G0PF:~$ ./page
Enter page number (0-3): 3
Page Fault!
Page must be loaded into memory.
chandrika_battina@DESKTOP-US3G0PF:~$ |

```

```
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address   = %d\n", virtual_address);
    printf("Page Number       = %d\n", page);
    printf("Offset              = %d\n", offset);
    printf("Frame Number        = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

```

chandrika_battina@DESKTOP x + v
chandrika_battina@DESKTOP-US3G0PF:~$ ls
ANU      file1.txt  fork1     fork3     hello     hel
Skill    fork      fork1.c   fork3.c   hello.c   inp
Week1.c  fork.c    fork2     fork4.c   hello.c.save out
example.txt fork.c.save fork2.c   forkk.c   hello.c.save.1 pid
chandrika_battina@DESKTOP-US3G0PF:~$ nano address.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc address.c -o address
chandrika_battina@DESKTOP-US3G0PF:~$ ./address
Value of x = 10
Virtual address of x = 0x7ffd1cbc88e4
chandrika_battina@DESKTOP-US3G0PF:~$ |

```

```

#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}

```

```

Physical Address - 150
chandrika_battina@DESKTOP-US3G0PF:~$ nano malloc.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc malloc.c -o malloc
chandrika_battina@DESKTOP-US3G0PF:~$ ./malloc
10 20 30 40 50
chandrika_battina@DESKTOP-US3G0PF:~$ |

```

```
chandrika_battina@DESKTOP  ×  +  ∨
GNU nano 8.7.1 null.c *
#include <stdio.h>

int main() {
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

```
chandrika_battina@DESKTOP-US3G0PF:~$ nano null.c
chandrika_battina@DESKTOP-US3G0PF:~$ gcc null.c -o null
chandrika_battina@DESKTOP-US3G0PF:~$ ./null
Before accessing memory
Segmentation fault ./null
chandrika_battina@DESKTOP-US3G0PF:~$ |
```